RK3572 MSMART User Guide

ID: RK-KF-YF-C69

Release Version: V1.0.0

Release Date: 2026-01-23

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a functional description and usage guidance for RK3572 VOP MSMART.

Product Version

Chip Models	Kernel Version
RK3572	LINUX Kernel 6.12

Target Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision Record

Version	Author	Date	Modification Description
V1.0.0	Zhang Yubing	2026-01-23	Initial Version

Table of Contents

RK3572 MSMART User Guide
1. Overview
2. MSMART Feature
3. MSMART Usage Constraints
   3.1 Alignment Requirements for Layer Formats
   3.2 Minimum Length Requirement for Layer act_width
   3.3 Constraints for Multi-Grids Usage
4. Driver Configuration Guide
   4.1 Single Layer Mode
   4.2 Multi-Grids Mode
   4.3 Driver Configuration Precautions
5. Multi-Grids Mode Functional Introduction
   5.1 Multi-Grids Spatial Layout Introduction
   5.2 Multi-Grids Layer Format Introduction
   5.3 Multi-Grids Scaling and Mirroring Introduction
6. User Space Interface Introduction

1. Overview

The MSMART layer is primarily a new layer introduced for NVR application scenarios, supporting both single-layer and multi-grids working modes. The following text will provide a detailed introduction to the functional features and usage requirements of the MSMART layer, as well as an explanation of the user space interface.

2. MSMART Feature

Supported Formats for Layers:

RGB: ARGB8888, RGB888, RGB565, ARGB2101010, ARGB1555

YUV: YUV420(8,10bit), YUV422(8,10bit), YUV444(8,10bit), YUV400(8,10bit)

MSMART layers are used to support the display of multi-grids, with two working modes: single-layer mode and multi-grids mode (multi-grids when the display area exceeds 1).

On RK3572, MSMART0 supports 4K, 8 x 8 multi-grids displays, which means up to 64 grids, with a maximum of 8 grids horizontally and 64 grids vertically.

On RK3572, MSMART1 supports 2K, 6 x 6 multi-grids displays, which means up to 36 grids, with a maximum of 6 grids horizontally and 36 grids vertically.

Note: All RGB and YUV layer formats do not support compression and TILE.

3. MSMART Usage Constraints

3.1 Alignment Requirements for Layer Formats

	YUV422	YUV420	YUV444_10bit	YUV422_10bit	YUV420_10bit	Other
act_width	2pixel	2pixel	2pixel	2pixel	2pixel	\
act_height	\	2line	\	\	2line	\
act_xoffset	2pixel	2pixel	4pixel	4pixel	4pixel	\
act_yoffset	\	2line	\	\	2line	\

3.2 Minimum Length Requirement for Layer act_width

Format	act_width
ARGB8888/ARGB2101010	>=16
RGB888	>=16
RGB565/ARGB1555	>=16
YUV_8bit: YUV_444/YUV422/YUV420/YUV400	>=17
YUV_10bit: YUV_444/YUV422/YUV420/YUV400	>=16

3.3 Constraints for Multi-Grids Usage

1. Multi-grids within the msmart layer cannot overlap spatially;
2. msmart0 can have at most 8 display grids in the same row, while msmart1 can have at most 6 display grids in the same row;
3. msmart0 can have a maximum of 64 grids, and msmart1 can have a maximum of 36 grids;
4. All layers of msmart multi-grids must maintain the same format and do not support setting different layer formats;
5. msmart multi-grids do not support individual grid scaling, only overall scaling is supported, with a reduction factor less than 2, and the magnification factor limit is the same as in single-layer mode (not exceeding 8 times);
6. msmart does not support x mirror, but supports overall y mirror.

4. Driver Configuration Guide

4.1 Single Layer Mode

In Single Layer Mode, layer information is configured to the corresponding registers through the AHB bus. The specific registers are as detailed in the table below:

Registers	description
WIN_CTRL(0x00)	\
WIN_ACT_INFO(0x1c)	win_act_width[12:0] win_act_height[28:16]
WIN_DSP_INFO(0x20)	win_dsp_width[12:0] win_dsp_height[28:16]
WIN_DSP_ST(0x24)	win_dsp_xst[12:0] win_dsp_yst[28:16]
WIN_GRID0_YRGB_MST(0x50)	grid0_yrgb_mst[31:0]
WIN_GRID0_CBCR_MST(0x54)	grid0_cbcr_mst[31:0]
WIN_GRID0_VIR(0x58)	grid0_vir_stride[15:0] grid0_vir_stride_uv[31:16]
WIN_GRID0_ACT_INFO(0x5C)	grid0_act_width[12:0] grid0_act_height[28:16]
WIN_GRID0_ACT_OFFSET(0x60)	grid0_act_xoffset[1:0] grid0_act_yoffset[17:16]
WIN_GRID0_DSP_OFFSET(0x64)	grid0_dsp_xoff[12:0] grid0_dsp_yoff[28:16]

The configuration of WIN_ACT_INFO and WIN_GRID0_ACT_INFO should be consistent;

The configuration of WIN_GRID0_ACT_OFFSET should be set to 0;

The configuration of WIN_GRID0_DSP_OFFSET should be set to 0;

WIN_DSP_ST represents the actual offset on the screen;

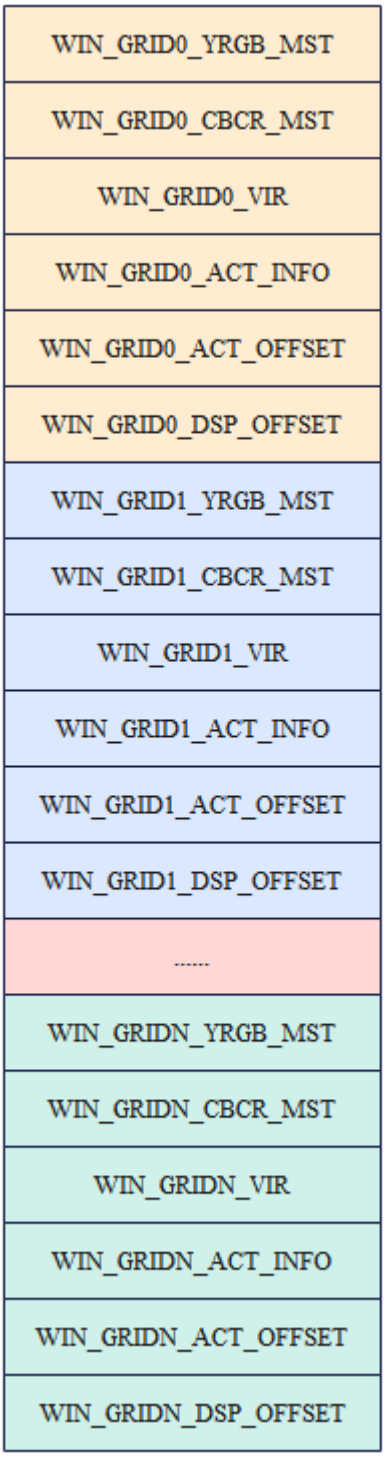
WIN_CTRL.win_num must be set to 0.

4.2 Multi-Grids Mode

1. The registers to be configured in Multi-Grids mode are similar to those in Single-Layer mode, including WIN_ACT_INFO(0x1c), WIN_DSP_INFO(0x20), WIN_DSP_ST(0x24). The GRID0 register is no longer required to be configured, and grid-related registers are directly obtained from DDR via the AXI bus;
2. When using Multi-Grids mode, users need to enable the `WIN_CTRL.win_multi_grid_en` signal, and at the same time, configure `WIN_CTRL.win_num`, where win_num is configured as the actual number of grids - 1, that is:

```
WIN_CTRL.win_num = real_num - 1, WIN_CTRL.win_multi_grid_en = 1.
```

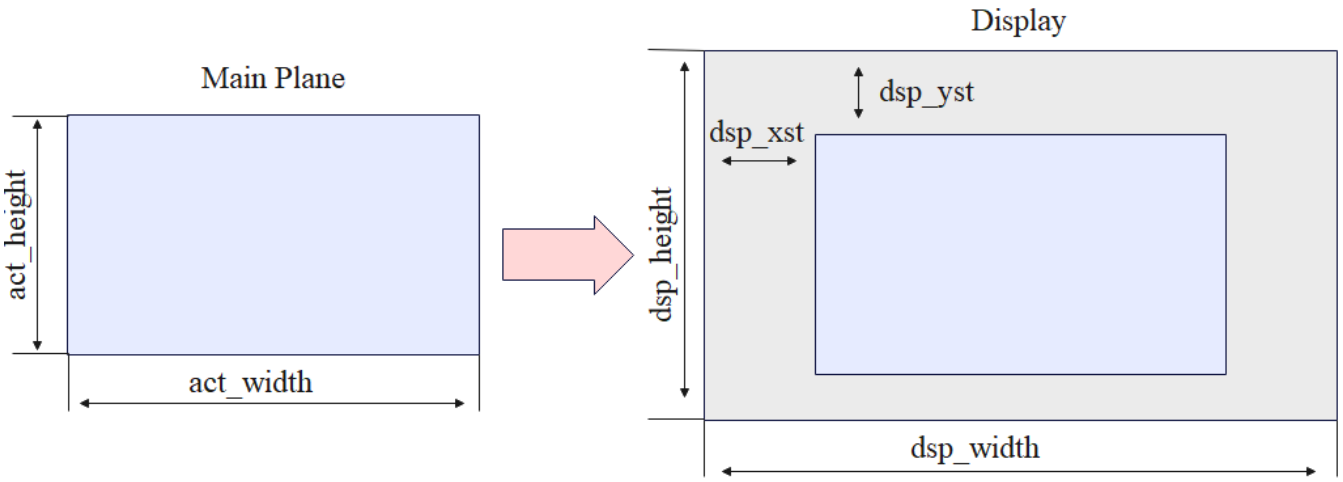
3. The display parameter information of Multi-Grids should be written into DDR in order (the first address must be aligned to 16 Bytes), such as the write-in information for GRID 0~N as follows:



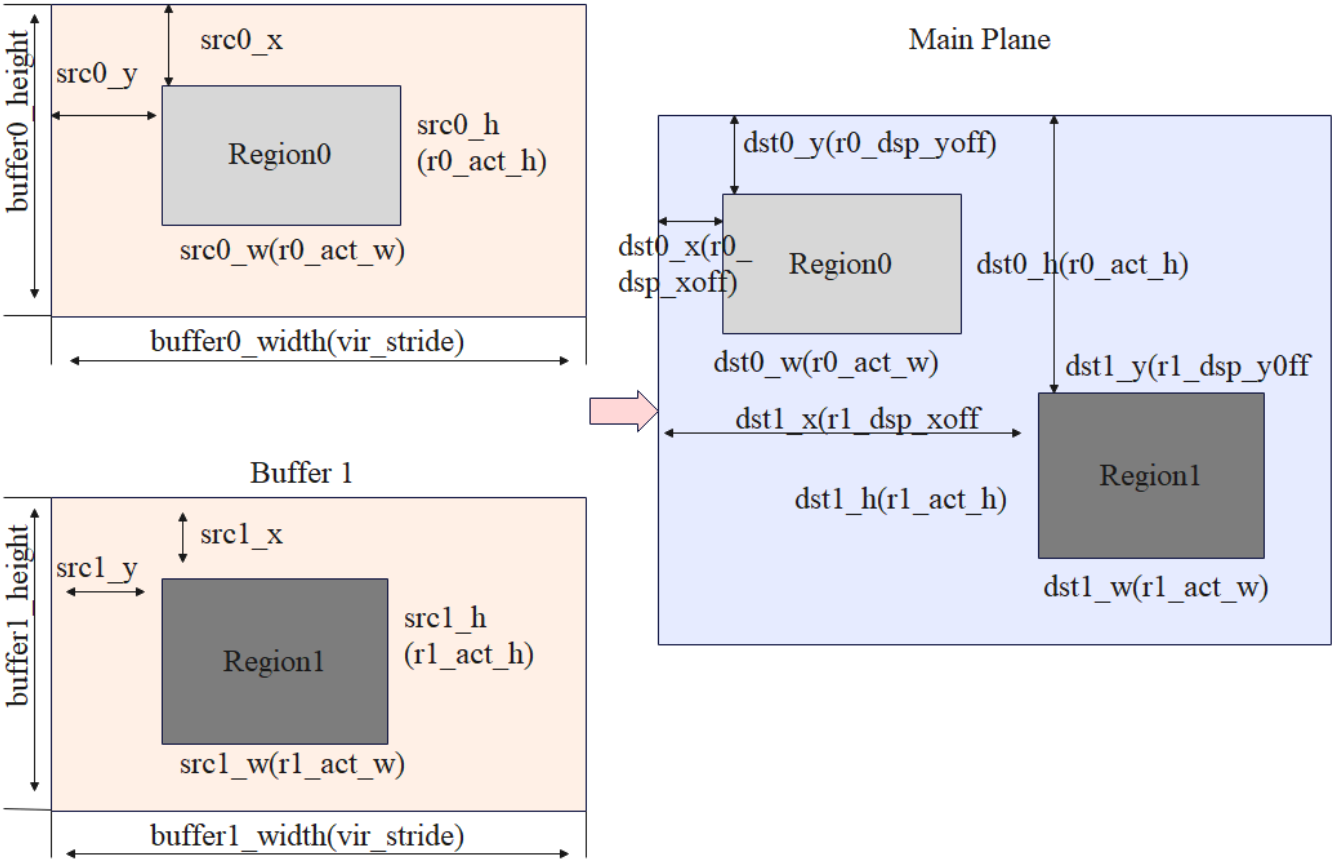
4. When organizing the display parameters of Multi-Grids into DDR, there is a requirement for size sorting, which requires sorting first by the starting vertical coordinate from smallest to largest, and then by the horizontal coordinate from smallest to largest;
5. After the msmart layer is started, it requires 3 rids (axi read id), and software needs to configure 2 rids, which are YRGB_RID and CBCR_RID. The other GRID_RID does not need to be configured by software, and the chip hardware defaults to `GRID_RID=YRGB_RID + 1`. Software's initial configuration should reserve GRID_RID;
6. Like the Single-Layer mode, the configuration setting for WIN_GRIDX_ACT_OFFSET is set to 0;

The parameter configuration for Multi-Grids can refer to the following schematic diagrams, which are divided into the configuration of the entire layer and the configuration of multiple internal grids.

Schematic diagram of the entire msmart layer configuration:



Schematic diagram of the configuration for multiple internal grids:



4.3 Driver Configuration Precautions

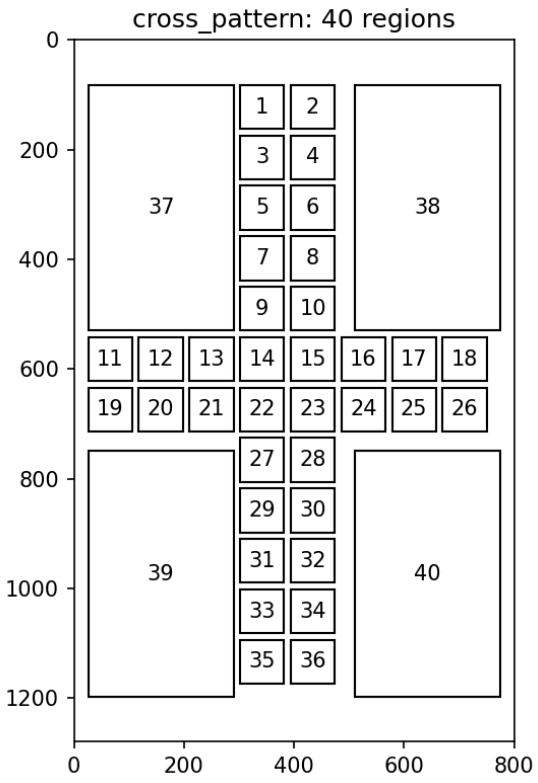
1. Multi-grids testing, grid layout switching, the width and height of the first grid will always maintain the width and height of the first area after power-on, updating the parameters of the first grid is ineffective.
   Problem Analysis: Not pulling up the soft reset may cause some internal logic not to be cleared cleanly.
   Solution: It is necessary to keep the soft reset bit of the layer pulled high.
2. Switching from multi-grids to a single layer, displaying ARGB8888 format will show anomalies, and

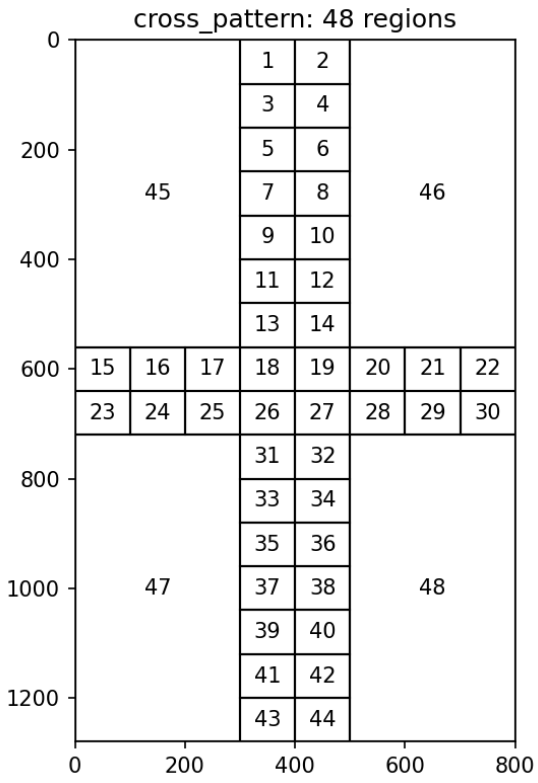
displaying NV12 will cause IOMMU error issues.
Problem Analysis: In the scenario of switching from multi-grids to a single layer, users need to first configure `WIN_CTRL.win_multi_grid_en = 0` before they can write to the display parameter area grid0 parameters (0x50-0x64) for single-layer mode via CPU.
Solution: Adjust the software configuration order, place the configuration of `WIN_CTRL.win_multi_grid_en = 0` before all grid0 configurations.

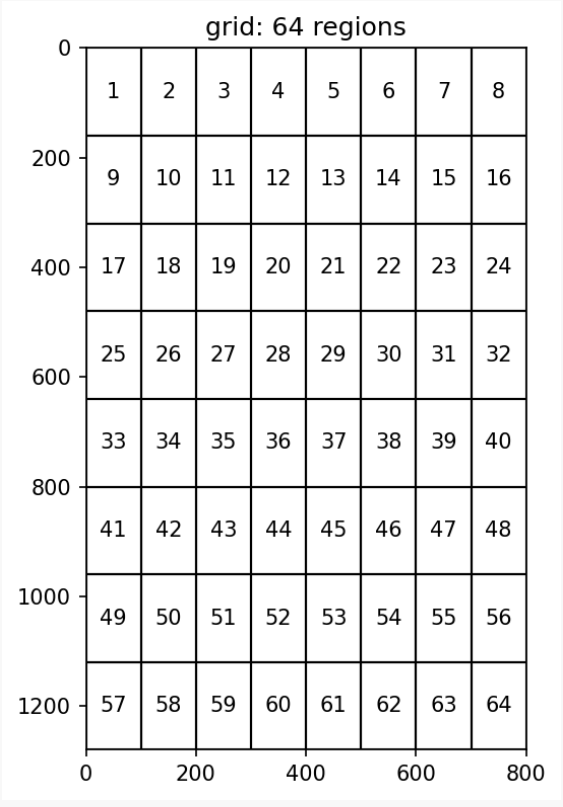
5. Multi-Grids Mode Functional Introduction

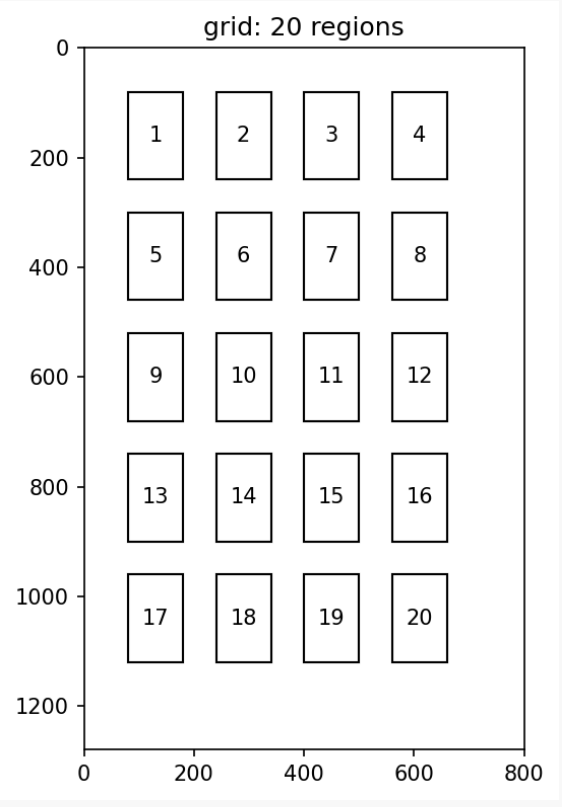
5.1 Multi-Grids Spatial Layout Introduction

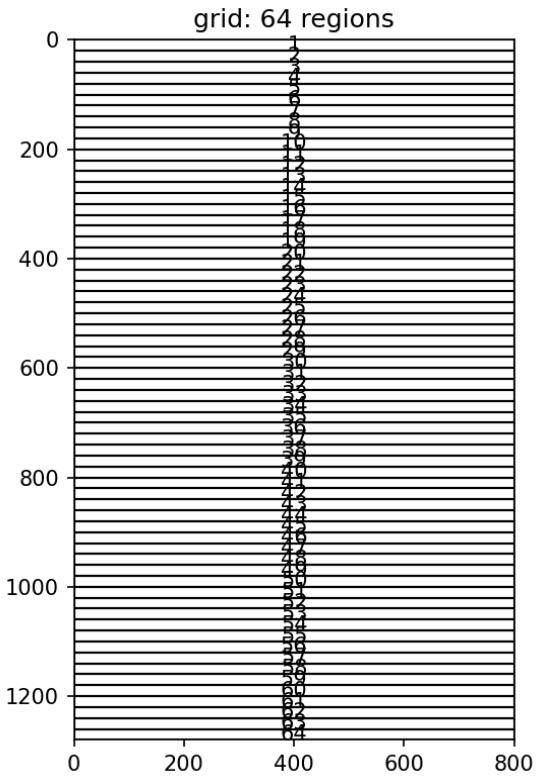
Below are the supported spatial layouts for testing:

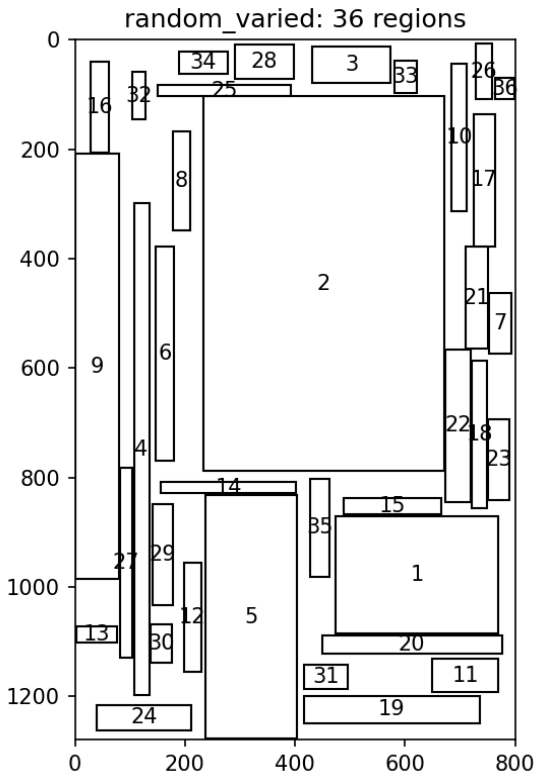

cross_pattern: 40 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
37
38
39
40
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36

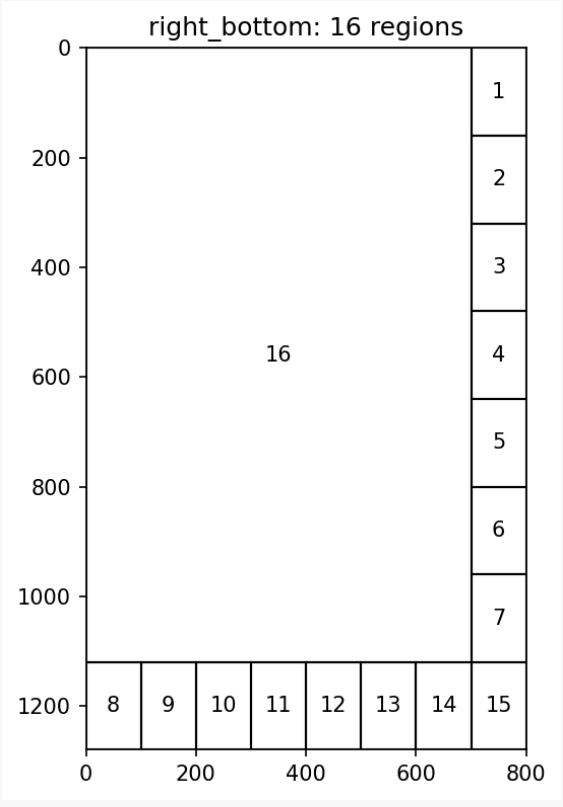

cross_pattern: 48 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
45
46
47
48
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44

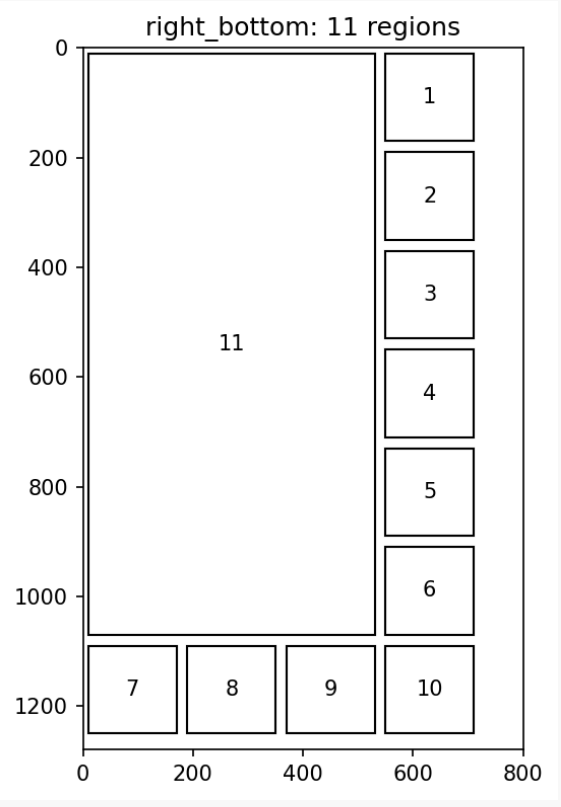

grid: 64 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64

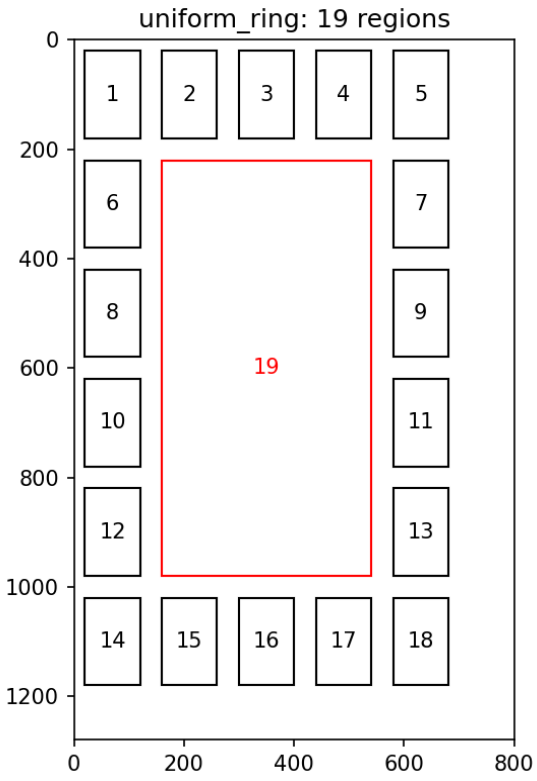

grid: 20 regions
0
200
400
600
800
1000
1200
0
200
400
600
800
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

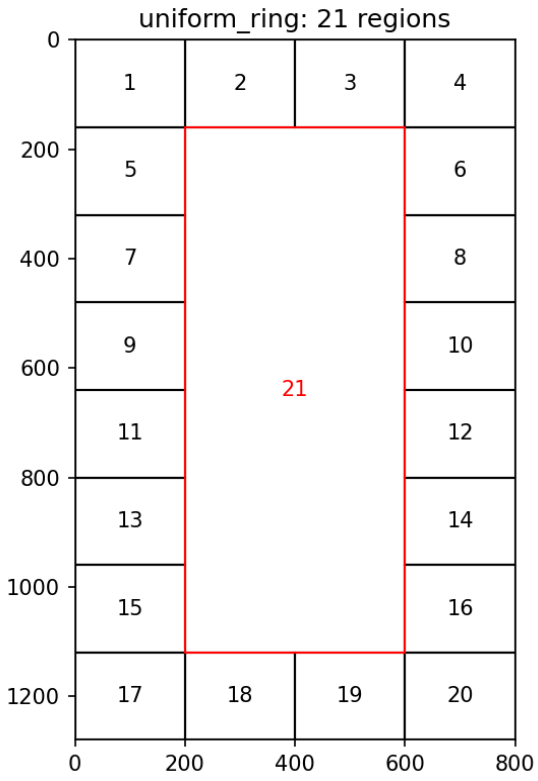

grid: 64 regions


random_varied: 36 regions


right_bottom: 16 regions


right_bottom: 11 regions





5.2 Multi-Grids Layer Format Introduction

Supports the following FourCC formats:

RGB: XR30, AR30, XB30, AB30, XR24, AR24, XB24, AB24, RG24, BG24, RG16, BG16, XR15, AR15, XB15, AB15

YUV: NV12, NV21, NV16, NV61, NV24, NV42, NV15, NV20, NV30

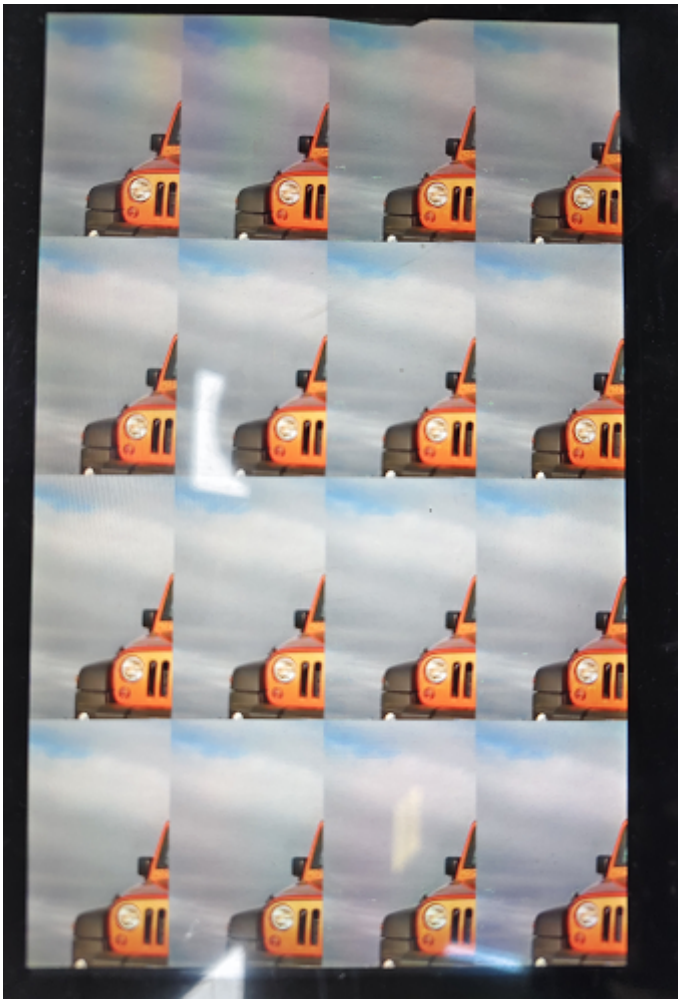
5.3 Multi-Grids Scaling and Mirroring Introduction

Scaling Down: The scaling factor should be less than 2; if the scaling factor equal to 2 will result in display anomalies.

Scaling Up: The magnification limit is consistent with a single layer.

Scaling effect is as follows:

Before Scaling

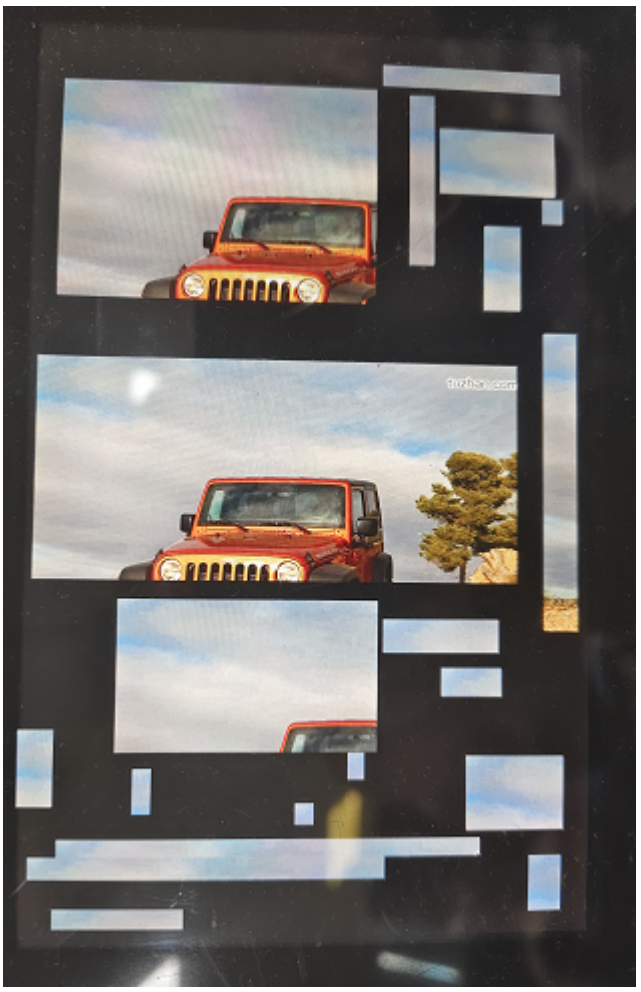


After Scaling



Supports Y-Mirror, the effect of Y-Mirror is as follows:

Before Y-Mirror



After Y-Mirror



6. User Space Interface Introduction

msmart only registers one plane, by providing a msmart layer blob attribute `MSMART_DATA`, which is used to pass the parameters of multiple grids. The structure of the passed parameters is as follows:

```
#define ROCKCHIP_VOP_MSMART_MAX_GRIDS_NUM      100
#define ROCKCHIP_VOP_MSMART_LUT_LENGTH         1536

/* msmart layer grid info */
struct rockchip_vop_msmart_grid {
        u32 fb_id; /* fb object contains grid format type */

        /* dest location which should not exceed the main layer */
        u32 dst_x;
        u32 dst_y;
        u32 dst_w;
        u32 dst_h;

        /* Source values are 16.16 fixed point */
        u32 src_x;
        u32 src_y;
        u32 src_h;
        u32 src_w;
```

```
        u32 reserved[20];
};

struct msmart_data {
        u32 version;

        /* Source values are 16.16 fixed point */
        u32 src_w;
        u32 src_h;

        u32 crtc_x;
        u32 crtc_y;
        u32 crtc_w;
        u32 crtc_h;

        u32 active_grid_num;
        u32 reserved[30];
        struct rockchip_vop_msmart_grid grid[ROCKCHIP_VOP_MSMART_MAX_GRIDS_NUM];
};
```

In which, `struct msmart_data` is the configured multi-grids information, with specific information as follows:

`version`: Version information, currently defaults to 0.

`src_w`: The width of the main area

`src_h`: The height of the main area

`crtc_x`: The x starting coordinate of the main area when outputting to the display

`crtc_y`: The y starting coordinate of the main area when outputting to the display

`crtc_w`: The width of the main area output

`crtc_h`: The height of the main area output

`active_grid_num`: The number of multi-grids currently configured

`reserved`: Reserved for future version expansion

`grid`: Multi-grids information

`struct rockchip_vop_msmart_grid` is the specific grid information:

`fb_id`: The fb_id of the grid buffer

`dst_x`: The x starting coordinate of the grid in the main area

`dst_y`: The y starting coordinate of the grid in the main area

`dst_w`: The width of the grid in the main area

`dst_h`: The height of the grid in the main area

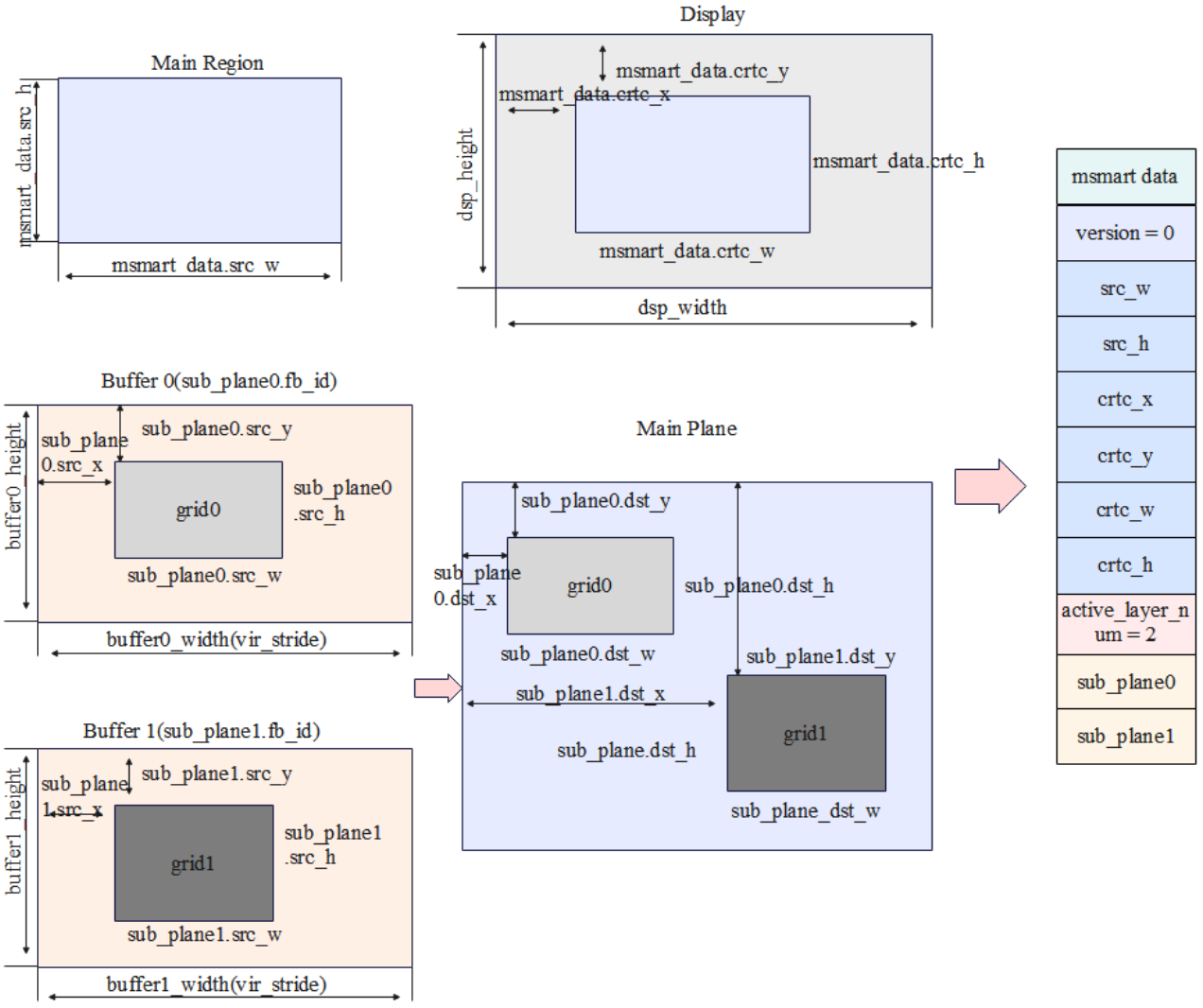
`src_x`: The x starting coordinate of the grid in the buffer

`src_y`: The y starting coordinate of the grid in the buffer

`src_w`: The width of the grid in the buffer

`src_h`: The height of the grid in the buffer

Currently, when the userspace application is used, the starting coordinates, width, height, fb_id, and other information of the multi-grids sub-layers and the overall layer information are passed to the kernel driver through the `MSMART_DATA` attribute. At the same time, the information bound by the drm plane bound to msmart is the information of the first grid.



Note:

1. When configuring multiple grids, applications do not need to sort them according to the starting x, y coordinates of each grid; the driver will automatically sort them;
2. The number of grids configured, the position of the grids, and the size of the grids can be dynamically changed for each frame;
3. When configuring scaling, only overall scaling is allowed, that is, the src_w/h and crtc_w/h of the main area, and the src_w/h and crtc_w/h of the grids need to be consistent;
4. When switching to single-layer mode, the `MSMART_DATA` attribute needs to be cleared at the same time;

Current ovltest usage:

```
static int set_msmart_plane(struct device *dev, struct plane_arg *p,
```

```
            struct msmart_sub_plane *msmart_plane,
            const char *file_name)
{
    uint32_t handles[4] = {0}, pitches[4] = {0}, offsets[4] = {0};
    unsigned int old_fb_id;
    struct bo *plane_bo;
    char name[PIC_NAME_MAX_LEN];
    int ret = 0;

    if (!strcmp(msmart_plane->file_name, "none"))
        strcpy(name, file_name);
    else
        strcpy(name, msmart_plane->file_name);

    plane_bo = msmart_plane->old_bo;
    msmart_plane->old_bo = msmart_plane->bo;

    if (!plane_bo) {
        plane_bo = ovl_bo_create(dev->fd, p->fourcc, p->afbc_en || p->rfbc_en,
                msmart_plane->vir_w, msmart_plane->vir_h,
                msmart_plane->vir_h, handles, pitches, offsets, name);
        if (plane_bo == NULL)
            return -1;
        ret = drmModeAddFB2(dev->fd, msmart_plane->vir_w, msmart_plane->vir_h, p->fourcc,
                handles, pitches, offsets, &msmart_plane->fb_id, 0);
        if (ret) {
            fprintf(stderr, "failed to add fb: %s\n", strerror(errno));
            return -1;
        }
    }
    msmart_plane->bo = plane_bo;

    old_fb_id = p->fb_id;
    msmart_plane->old_fb_id = old_fb_id;

    return 0;
}

static int set_msmart_planes(struct device *dev, struct plane_arg *p,
            struct msmart_data *msmart_data, const char *file_name,
            unsigned int offset)
{

    int i;
    int ret = 0;
    unsigned int blob_id = 0;
    struct msmart_sub_plane *msmart_plane;

    if (!p->msmart_arg.sub_layer_num)
        return 0;

    for (i = 0; i < p->msmart_arg.sub_layer_num; i++) {
        ret = set_msmart_plane(dev, p, &p->msmart_arg.msmart_plane[i], file_name);
```

```
        if (ret)
            return ret;
    }

    msmart_data->active_grid_num = p->msmart_arg.sub_layer_num;
    for (i = 0; i < p->msmart_arg.sub_layer_num; i++) {
        msmart_plane = &p->msmart_arg.msmart_plane[i];
        msmart_data->grid[i].fb_id = msmart_plane->fb_id;
        msmart_data->grid[i].dst_x = msmart_plane->dst_x;
        msmart_data->grid[i].dst_y = msmart_plane->dst_y;
        msmart_data->grid[i].dst_w = msmart_plane->dst_w;
        msmart_data->grid[i].dst_h = msmart_plane->dst_h;

        msmart_data->grid[i].src_x = msmart_plane->src_x << 16;
        msmart_data->grid[i].src_y = msmart_plane->src_y << 16;
        msmart_data->grid[i].src_w = msmart_plane->src_w << 16;
        msmart_data->grid[i].src_h = msmart_plane->src_h << 16;
    }
    drmModeCreatePropertyBlob(dev->fd, msmart_data, sizeof(struct msmart_data), &blob_id);
    p->msmart_arg.blob_id = blob_id;
    add_property_optional(dev, p->plane_id, "MSMART_DATA", blob_id);

    return 0;
}

static int atomic_set_msmart_plane(struct device *dev, struct plane_arg *p, const char *file_name,
                bool update, unsigned int offset)
{
    struct msmart_sub_plane *msmart_plane;
    struct msmart_data msmart_data = {0};
    struct crtc *crtc = NULL;
    int crtc_x, crtc_y, crtc_w, crtc_h;
    unsigned int i;
    int ret;

    /* Find an unused plane which can be connected to our CRTC. Find the
     * CRTC index first, then iterate over available planes.
     */
    for (i = 0; i < (unsigned int)dev->resources->res->count_crtcs; i++) {
        if (p->crtc_id == dev->resources->res->crtcs[i]) {
            crtc = &dev->resources->crtcs[i];
            break;
        }
    }

    if (!crtc) {
        fprintf(stderr, "CRTC %u not found\n", p->crtc_id);
        return -1;
    }

    if (!update)
        fprintf(stderr, "testing %dx%d@%s on plane %u, crtc %u\n",
            p->w, p->h, p->format_str, p->plane_id, p->crtc_id);
```

```
    if (p->rotation & (DRM_MODE_ROTATE_90 | DRM_MODE_ROTATE_270)) {
        crtc_w = p->crtc_w ? p->crtc_h : p->h * p->scale;
        crtc_h = p->crtc_h ? p->crtc_h : p->h * p->scale;
    } else {
        crtc_w = p->crtc_w ? p->crtc_w : p->w * p->scale;
        crtc_h = p->crtc_h ? p->crtc_h : p->h * p->scale;
    }
    if (!p->has_position) {
        /* Default to the middle of the screen */
        crtc_x = (crtc->mode->hdisplay - crtc_w) / 2;
        crtc_y = (crtc->mode->vdisplay - crtc_h) / 2;
    } else {
        crtc_x = p->x;
        crtc_y = p->y;
    }
    msmart_data.src_w = p->w << 16;
    msmart_data.src_h = p->h << 16;
    msmart_data.crtc_x = crtc_x;
    msmart_data.crtc_y = crtc_y;
    msmart_data.crtc_w = crtc_w;
    msmart_data.crtc_h = crtc_h;

    ret = set_msmart_planes(dev, p, &msmart_data, file_name, offset);
    if (ret)
        return ret;

    msmart_plane = &p->msmart_arg.msmart_plane[0];

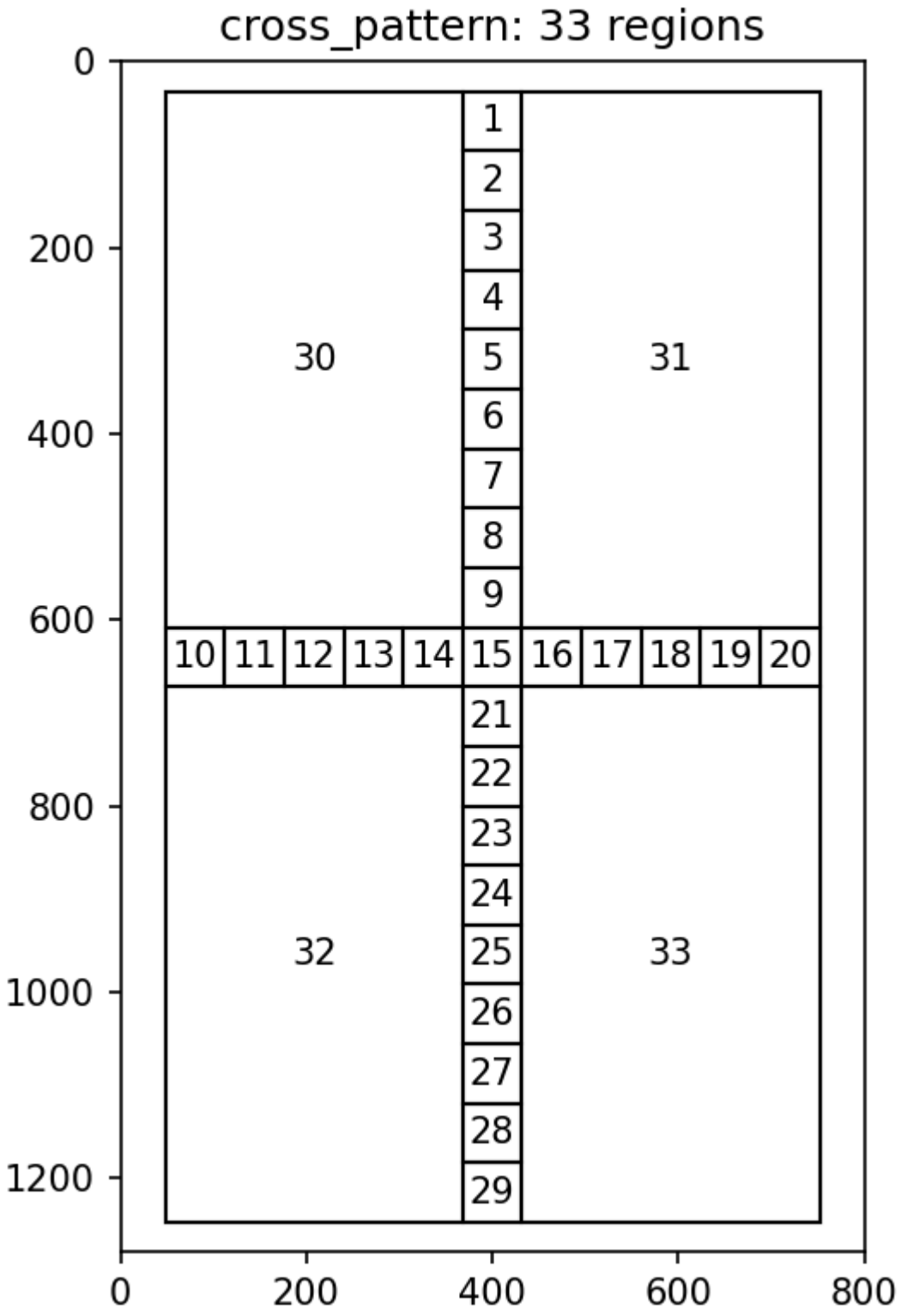
    p->old_bo = msmart_plane->old_bo;
    p->bo = msmart_plane->bo;

    add_property(dev, p->plane_id, "FB_ID", msmart_plane->fb_id);
    add_property(dev, p->plane_id, "CRTC_ID", p->crtc_id);
    add_property(dev, p->plane_id, "SRC_X", offset << 16);
    add_property(dev, p->plane_id, "SRC_Y", offset << 16);
    add_property(dev, p->plane_id, "SRC_W", (msmart_plane->src_w - offset) << 16);
    add_property(dev, p->plane_id, "SRC_H", (msmart_plane->src_h - offset) << 16);
    add_property(dev, p->plane_id, "CRTC_X", msmart_plane->dst_x);
    add_property(dev, p->plane_id, "CRTC_Y", msmart_plane->dst_y);
    add_property(dev, p->plane_id, "CRTC_W", msmart_plane->dst_w);
    add_property(dev, p->plane_id, "CRTC_H", msmart_plane->dst_h);
    add_property(dev, p->plane_id, "rotation", p->rotation);
    add_property(dev, p->plane_id, "zpos", p->zpos);
    if (p->no_premulti)
        add_property(dev, p->plane_id, "pixel blend mode", 1);
    else
        add_property(dev, p->plane_id, "pixel blend mode", 0);
    add_property(dev, p->plane_id, "EOTF", p->eotf);
    add_property(dev, p->plane_id, "COLOR_SPACE", p->colorspace);

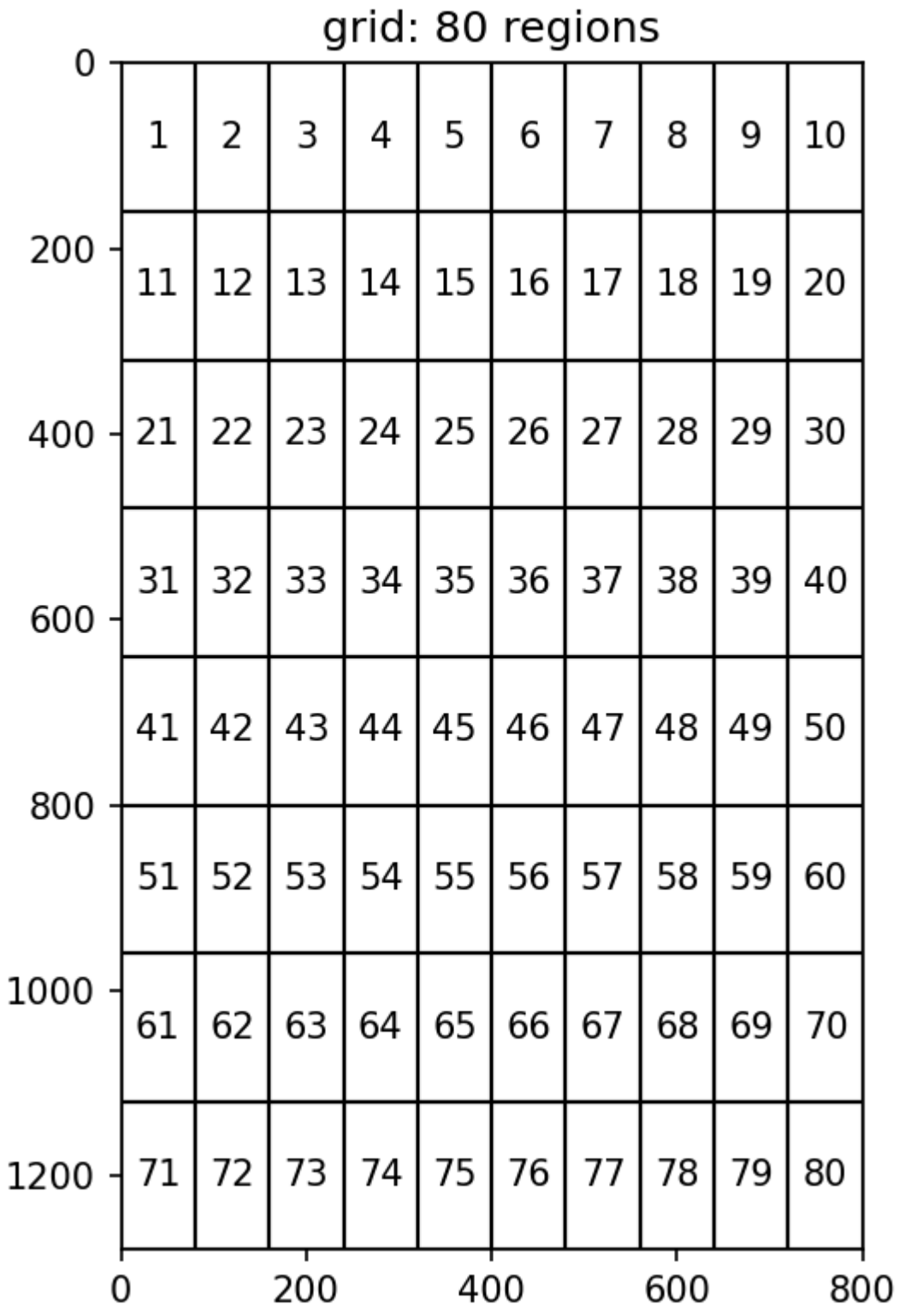
    return 0;
}
```

msmart layer unsupported cases:

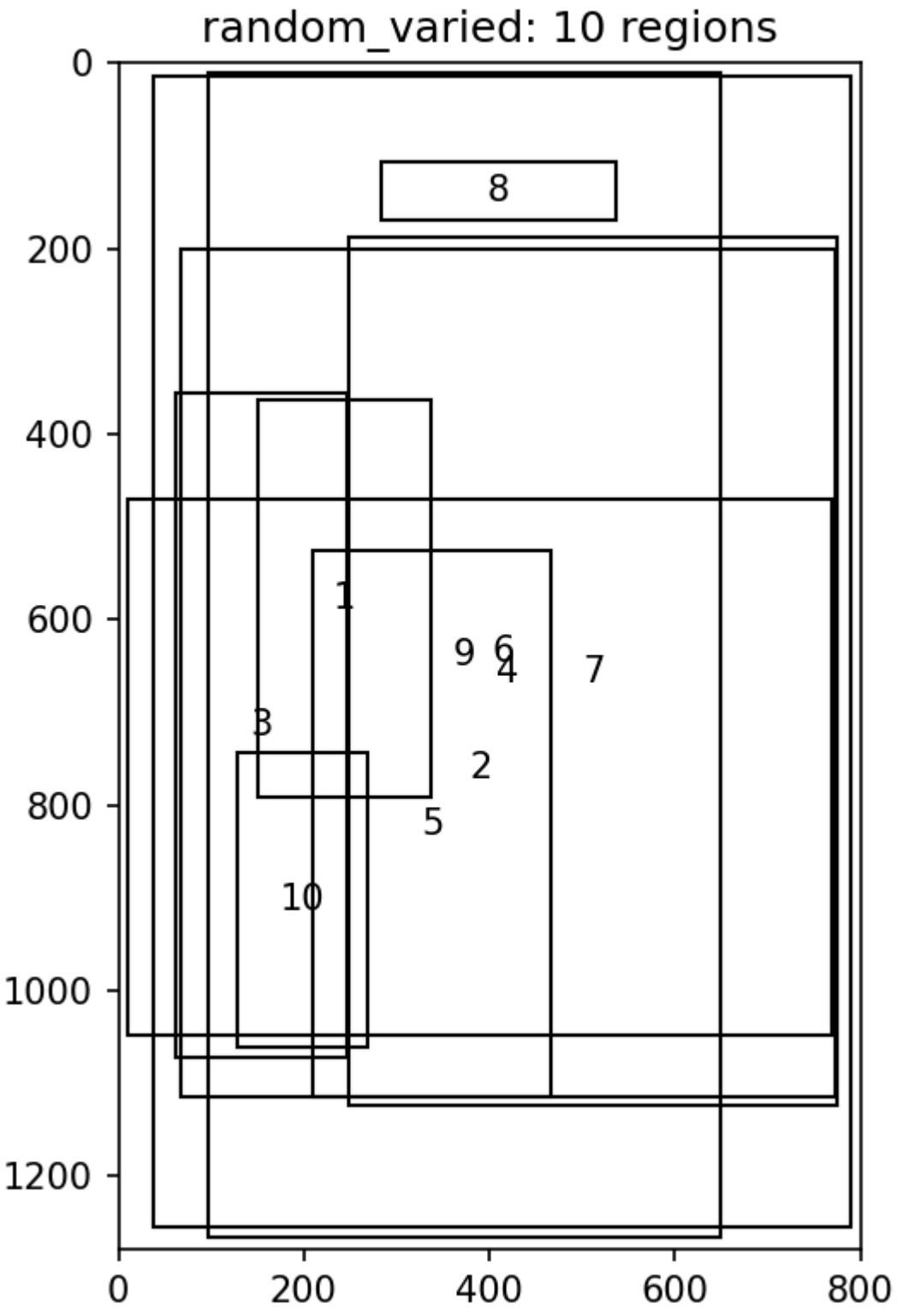
Exceeding the maximum grid limit per row:



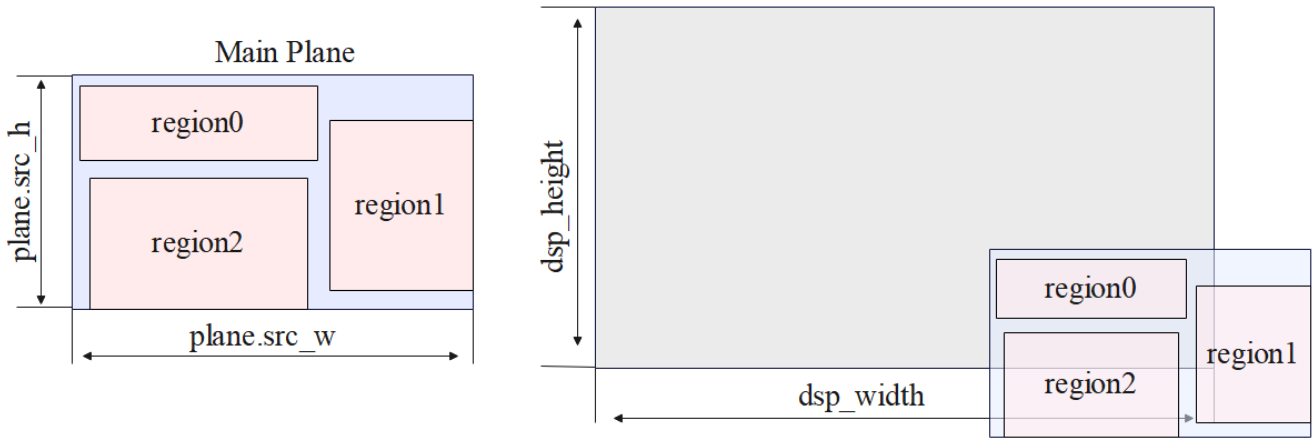
Exceeding the maximum grid limit per row and the maximum quantity limit:



Grids overlapping:



Exceeding the screen display range:


Main Plane
region0
region1
region2
plane.src_h
plane.src_w
dsp_height
dsp_width
region0
region1
region2